

# Supporting Decisions in a Near-Unpredictable Game

---

A technical description of ABRA, a decision-support model family for competitive Pokémon

Version 3.59.0 · Last updated 2026-08-06 Will Hooper · ABRA

*This is a living document, updated in the same pass as any change to the code, together with the deck and the technical documentation. A prior conclusion is never silently rewritten; new information is added and what changed is stated. See `CHANGELOG.md`.*

---

## Abstract

---

ABRA is a decision-support model family for **Pokémon Champions VGC, Regulation M-B, best-of-one closed-sheet ladder**. It continuously ingests public battle replays from Pokémon Showdown and stores the durable facts of every game, then builds small, CPU-trainable models on that store. Its central empirical finding governs its design: **predicting the winner of a game from the two team sheets is near-impossible in this format — even a player-Elo model ties a coin**. ABRA therefore does not sell outcome prediction. It follows the recipe that worked in poker, Diplomacy, and sports analytics: *support decisions, don't predict outcomes*, and judge every model by a proper score against an honest baseline with a confidence interval. **As of 3.59.0 the headline metric is exploitability rather than win rate** (§0, ADR-003): VGC is formally an imperfect-information game, the only prior work in this exact format measures its own agents at approximately 100% exploitable despite beating a professional, and the thesis under test is that a per-turn re-solving agent is harder to exploit than a compiled policy — *unknown*, and stated as the experiment rather than the assumption. This paper states that thesis and its metric, the empirical ceiling, the data model, each model with its validated result (including two honest negatives), the mathematics, the limits, and the road to the in-battle engine (ALAKAZAM).

## 0. The thesis, and the metric that follows from it (3.59.0)

---

**This project's headline metric is exploitability, not win rate.** The change is recorded in ADR-003 and it is forced by a measurement somebody else made.

**The field has been treating VGC as a chess problem or a pure-RL problem. It is a poker problem.** Chess and Go are perfect-information games: there is one true state, both players see it, and minimax or MCTS over that state is sound. VGC is not. A player does not know which four of the opponent's six will be brought, their items, their abilities, or the fourth move on each set, and that hidden information is not noise to be averaged away — it is *strategically exploitable*. The correct solution concept is therefore a **Nash equilibrium in mixed strategies, not a single best move**, which is poker's situation verbatim and the reason RL+search methods that are sound in chess break in imperfect-information games (ReBeL, ref. 4). `docs/POKER-TO-POKEMON.md` works the correspondence through term by term and is honest about the three places it breaks — simultaneity, action-space and horizon scale, and the nature of chance.

That argument was made from theory and had no measurement behind it. **VGC-Bench supplies the measurement.** Angliss, Cui, Hu, Rahman and Stone (AAMAS 2026, ref. 5) trained behaviour cloning on 700,000+ human battle logs and fine-tuned with PPO under self-play, fictitious play and double oracle. In a single-team mirror match their agent **beat a World Championships competitor**. And in the same paper:

- "In almost all cases, **all agents are approximately 100% exploitable**" — measured by training a best-response policy against each agent.
- Their expert tester's feedback: "*although the agent is strong on initial play, it does have noticeable dips in performance in certain states. **After enough successive games, strong human players can adapt and beat the agent.***"
- Against their *advanced* (not expert) tester the agent won **2 of 5**.

**That is not a weakness of their execution. It is the predicted behaviour of a compiled policy** — a fixed map from state to action — in an imperfect-information game. A best response can find and drill its blind spots, and so can a human given five games. Poker learned this over 2007–2021 and answered it with equilibrium mixing and continual re-solving (CFR, DeepStack, Libratus, ReBeL; refs. 1–4).

**The thesis is therefore that a re-solving agent should be harder to exploit than a compiled one.** A learned policy *recalls*; a search *recomputes*. A best-response exploiter attacks a fixed mapping, and a per-turn re-solve presents none. **Whether this survives simultaneity, stochasticity and a ~6-turn horizon is UNKNOWN — that is the experiment of this project, not its assumption.**

**Two consequences for the model family.** WOBBUFFET (§4, and `docs/MODELS.md`) moves from side-check to primary instrument, with VGC-Bench's ~100% as the published comparator. SLOWKING stops being "the preview solver" and becomes the shape of the whole agent.

**And the honest state of that metric today is that we do not have one.** `data/exploitability.json` is declared void; the 2026-07-26 figure was fitted on 17 features against the 58 we ship and the 2026-08-04 re-run had its defender refitted while it was running. Leading with a metric this project cannot currently produce is deliberate — it makes the gap a deliverable instead of a footnote.

### 0.1 Why the comparison is legitimate although the agents can never meet

VGC-Bench's public checkpoints are Regulation M-A; we are Regulation M-B, and their own paper shows policies do not transfer across team sets. A head-to-head is impossible. But **exploitability is intrinsic**: it is defined against a best response trained against *you*, in *your* format, so the two numbers live on the same scale without the two agents ever playing a game.

Two further points sharpen rather than weaken the frame:

- **VGC-Bench is open team sheets** — the same information setting as our Reg M-B best-of-three. They had *more* information than a closed-sheet agent and were still ~100% exploitable. The exploitability comes from holding a fixed policy, not from hidden teams.
- **Their dataset is not usable by us, and this project's code already said so.** Their Reg M-B holding is 4,167 games over 4 days in June 2026, and all 4,167 are already in our store as `data/games.ots.jsonl`, against our own 9,701 best-of-three games over 15 days. The 700,000 headline is Reg M-A, the previous regulation. An earlier claim in this project that their archive covered our format inferred coverage from a *filename* and is withdrawn; `docs/PRIOR-ART.md` §2 carries the correction.

### 0.2 What we are not claiming

- **Not that we will beat VGC-Bench.** Their agent beat a Worlds competitor; ours has never played a human.
- **Not that search is known to work here.** Metamon (RLC 2025, ref. 5) reached top 10% in singles with no search at all, and Future Sight AI removed its machine learning entirely after finding a structural method beat it on both accuracy and speed. Both are live counter-evidence.
- **Not novelty on the format or the infrastructure.** VGC-Bench owns both, including the poke-env doubles support the field now uses. We are not first, and in our own format we are behind.

**If the thesis fails, the instrumentation still stands.** No project in `docs/PRIOR-ART.md` publishes a mechanics census that must be shown red before it counts, a step-level protocol differential against the official engine, ratchets on silent failure, or a record of what it retracted. If search loses, this remains the only account of what a hand-written VGC simulator gets wrong and how you would know — publishable precisely because everyone else avoided the problem by not having it.

**0.3 The engine is justified if and only if search pays**

VGC-Bench used real Showdown through poke-env and carried **no engine-correctness debt at all**, because behaviour cloning and PPO do not need a fast simulator: they need throughput at training time, not at decision time. We wrote MEDICHAM (§3) so that **per-turn re-solving is affordable**. That makes the engine work falsifiable rather than assumed, and it promotes one roadmap item to the status of a project gate. Supporting evidence that this is the real trade and not a rationalisation — every project that searches hits the engine-speed wall, and the pattern is clean:

| project         | searches? | engine                      | depth reached                |
|-----------------|-----------|-----------------------------|------------------------------|
| VGC-Bench       | no        | real Showdown               | n/a                          |
| Future Sight AI | yes       | modified Showdown           | ~3 turns in 15 s on 16 cores |
| Foul Play       | yes       | built its own (poke-engine) | ~10+ turns                   |

**The plan is four phases, and the fourth is a result rather than a defeat:**

|   |                                                                                  |                                                    |
|---|----------------------------------------------------------------------------------|----------------------------------------------------|
| 1 | finish MEDICHAM                                                                  | search needs an engine that is fast AND correct    |
| 2 | GATE #62                                                                         | does compute buy anything: untimed vs on-the-clock |
| 3 | if yes -> search, and measure EXPLOITABILITY against their ~100%                 |                                                    |
| 4 | if no -> adopt their recipe: BC + PPO self-play/FP/DO, open source, reproducible |                                                    |

Phase 4 is cheap precisely because VGC-Bench made it so — the method is published, open-source and reproducible — and taking it would be a finding about VGC, not a failure of this project.

**On compute.** Cores help the search (it is CPU-bound and root-parallelisable); GPUs help behaviour cloning and PPO. MILTANK currently needs **26 s against a 20 s budget on one core of sixteen**, so sixteen cores fixes the clock today. But root parallelisation scales **sublinearly**, so cores convert a failed budget into a met one rather than a shallow search into a deep one. Buying cores does not buy depth, and the phase-2 gate is about depth.

**1. The empirical ceiling (why the design is what it is)**

On 600+ held-out real Champions games, a Bradley-Terry player-Elo model reaches a held-out log-loss of **0.687 against a coin's 0.693** — a real but negligible edge. **A 2026-07-25 re-measurement makes the ceiling lower still:** the previously published "higher-rated player wins 55.0%" was computed with a name-only bot filter that missed six high-volume accounts. Removing them gives **52.4%, 95% CI [49.9, 54.9]** — an interval containing a coin flip. A cloned-policy rollout engine (MEDICHAM) does *worse* than a coin as a raw win-predictor.

**Re-measured 2026-08-04 on 6,886 clean games**, against the leaves MILTANK actually calls rather than the `winProb2` entry point the earlier readings scored. Paired against a coin on identical turn-0 positions, the in-game leaf ( `explore=1.0` , 200 rollouts, held-out n=1,378) loses by **Brier +0.0502, 95% CI [0.0371, 0.0628]**, and the team-preview leaf (n=6,886) by **+0.0740 [0.0668, 0.0813]**; both also lose to player-Elo. The reliability curve is nearly flat — the in-game leaf's 90-100% bucket wins 53.6% and its 0-10% bucket wins

53.8% — and it names the winner on **50.99% of 1,314 decisive calls, 95% CI [48.3, 53.7]**, which is a coin. The preview leaf discriminates barely: 53.22% of 6,700 (CI [52.0, 54.4]), about 1.9 points above its own split-half noise floor.

*This supersedes, and partly corrects, the earlier reading.* The 2026-07-23 figure ("log-loss  $\approx 1.2$ ; picks the winner on  $\sim 44\%$  of decisive calls, i.e. systematically **inverted**") is retained here because a prior conclusion is never silently rewritten, but the inversion **does not replicate**: at twenty times the sample the same family of leaves sits slightly *above* chance, not below it. The 44% was a small-sample excursion. What replicates, and replicates decisively, is the **overconfidence**: the preview leaf puts 25.6% of its predictions into the two extreme buckets and is wrong there by about 40 points. Full curve, counts and intervals in `data/winrate-backtest.json`.

The conclusion is not "our models are weak." It is a property of the game: a two-player, zero-sum, **imperfect-information, simultaneous-move** game with a non-transitive metagame has an irreducible outcome-prediction ceiling from team sheets alone. This is the same reason expected-goals (xG) models in football predict *shot quality* rather than final scores. **Design consequence: stop predicting outcomes; support decisions.** Everything below serves that.

## 2. Data: store raw, analyse on top

ABRA reads Showdown's public replay API ( `search.json?format=` , `search.json?user=` , `<id>.log` ); it reads nothing private and creates no accounts ( `SECURITY.md` ). The extractor ( `engine/durable-ingest.js` , `extract()` ) turns one battle log into one durable record:

| Field                               | Meaning                                                            |
|-------------------------------------|--------------------------------------------------------------------|
| <code>id</code> , <code>date</code> | replay id and upload time                                          |
| <code>p1</code> , <code>p2</code>   | <code>{name, rating, bot}</code> per player                        |
| <code>six.p1/p2</code>              | the revealed team of six                                           |
| <code>brought.p1/p2</code>          | the four actually brought                                          |
| <code>lead.p1/p2</code>             | the two led                                                        |
| <code>sets</code>                   | per species, the moves / item / ability the replay <i>revealed</i> |
| <code>turns</code>                  | per-turn events (moves, damage, faints, status, field)             |
| <code>winner</code>                 | the winning name                                                   |

The store is append-only JSON Lines keyed by replay id: idempotent, deduplicated at read time, and grown hourly by a GitHub Action. The **governing rule** is *store raw, analyse on top*: every filter (rating tier, humans-only, archetype, playstyle) is a re-computation over the store, never a re-pull. Changing how we segment games is free; the fetch is a one-time cost. About 2,600 public games/day are available, and the store grows  $\sim 18\text{k}/\text{week}$ , so every model below sharpens on its own over time.

## 3. The validated foundation — exact damage (MEDICHAM)

The one component that is *not* a coin flip is the damage engine. MEDICHAM's Gen-9 doubles damage pipeline ( `engine/medicham2-browser.js` ) is validated against the Smogon damage calculator (the community ground-truth) on 31 meta scenarios: **within 5% on 100% of scenarios, median error 0%, worst 3%** (16-roll rounding). This is gated in CI ( `engine/validate_damage.js`  $\rightarrow$  `data/damage-validation.json` ). Every model that reasons about damage builds on this, and "will this move KO?" is a *winnable* prediction, unlike "who wins the game."

### 3.0 Why a hand-written engine exists at all, and the corrected speed figure (3.59.0)

**ADR-001 decided this architecture on a benchmark of 29 against 3,401 battles/sec/core — a ratio of 117x — and that ratio does not reproduce.** Re-measured on the same machine, both engines on the same four teams (derived from the store rather than typed), 8-second runs at a 60-turn cap:

|               | turns/sec | battles/sec |
|---------------|-----------|-------------|
| MEDICHAM      | 13,041    | 217         |
| champions_sim | 523       | 28          |
| ratio         | 24.9x     | 7.7x        |

turns/sec **is the comparable unit and** battles/sec **is not.** The two engines were driven differently — MEDICHAM to its 60-turn cap, Showdown with `choose('default')` to a natural end — so a "battle" is not the same amount of work on the two sides, and the 7.7x is not like-for-like. The honest statement of the gap is **24.9x**. The July figures are retained above and in ADR-001 because a prior conclusion in this project is never silently rewritten, and a third reading exists that is neither: ROADMAP #61 measured MEDICHAM at 1,606 battles/sec. **Nothing ratchets engine speed**, which is how three readings of one quantity can differ by an order of magnitude with no test going red.

**The architectural decision survives the correction, but its justification changes.** A 24.9x gap still rules out live browser simulation, so ADR-001's conclusion stands. What no longer stands is "117x" as the reason. The reason is now the one §0.3 gives and it is falsifiable: **the engine work is justified if and only if search pays**, gated by ROADMAP #62.

#### 3.1 The engine can now say WHAT it did, not only what state it reached (3.58.0)

The damage validation above, the 150-row differential and the five scripted whole-game comparisons all compare **outcomes**: a number, or a state after a turn. None of them can see an ordering, and none can say *which mechanism* produced a disagreement. Showdown's own protocol log can — it is a step-level trace already labelled with the mechanism behind each decision ( `| -unboost|` is a stat drop, `| -enditem|` is an item being spent, the order of two `|move|` lines is turn order).

`engine/medicham2-browser.js` now emits that stream on request. The event set is **derived from Showdown's** `add()` **call sites** rather than transcribed ( `engine/derive_protocol_events.js` → `data/protocol-events.json`, whose own `showdownEvents`, `emittedCount`, `notEmittedCount` and `partialCount` read 91 / 36 / 58 / 10), and two gates fail the run — claiming an event Showdown never emits, or leaving one unexplained. The scan reads this **format's** overrides, not the generic protocol: Champions emits `| -supereffective|POKEMON|N` where the base engine emits two fields.

It changes no mechanic, and two things it found on its first night are recorded because they are about **what the existing instruments cannot see**:

- 1. The damage differential is an endpoint comparison.** It calls the reference at `roll=0` and `roll=15` against MEDICHAM's `min` and `max`. In between, MEDICHAM interpolates linearly over an 11-integer range and samples it uniformly; Showdown floors sixteen base values separately. 149/150 endpoint agreement is compatible with every interior roll being off by one or two, and with every roll's *probability* being wrong. This is a limitation of the measurement, stated; it is not a claim that the damage is wrong.
- 2. Order within a hit differs and end-of-turn state does not.** MEDICHAM resolves the knock-off, the resist berry and the contact punish before subtracting the target's HP. The whole-game state comparison agrees on every turn of all five scripted games; the trace does not agree on the order.

Neither is fixed here. Changing how a damage roll is drawn moves every seeded run in the repository.

## 4. The models and their validated results

---

Every probability ships a **proper score** (log-loss and/or Brier), a **confidence interval** (clustered by game where states within a game are correlated), and an **honest baseline**, persisted to JSON and gated in CI.

### 4.1 GURU — meta / matchup matrix (descriptive)

From REAL outcomes, `engine/guru.py` builds an archetype × archetype matchup matrix (K is chosen from the data; see `data/archetypes.json`) over the generated game count in `data/live.js` (hardcoded sizes are retracted, S13) — games, each cell a win-rate with a **Wilson score interval**. GURU is *descriptive*: its own predictive test shows per-game winner prediction from the matrix ties a coin (log-loss 0.7122 vs 0.6931), exactly as §1 predicts. Its value is honest matchup structure with error bars, and it is the real (not simulated) payoff matrix that SLOWKING solves. Output: `data/guru-matchups.json`, `data/guru.js`.

### 4.2 XATU — opponent belief (modest, useful)

`engine/xatu.py` learns, per species, the set (item/ability/moves) usually run, and predicts the opponent's next move from state. On held-out human moves the behaviour-clone reaches **top-1 35.9% (CI 35.2–36.5)**, **top-3 71.6%**, cross-entropy 2.27 nats — beating a species-agnostic baseline (4.54) and uniform-over-moveset (2.91). A modest but real signal; human move choice has genuine entropy. Output: `data/xatu.json`, `data/xatu.js`; harness `engine/eval_policy.py` → `data/policy-eval.json`.

### 4.3 PORY — mid-game win probability (RETRACTED as a value net; it is material arithmetic)

The pivot's proof. `engine/pory.py` reconstructs per-turn board state (mons alive out of four, mean active HP, turn) and fits a logistic value net. Held-out, clustered by game: **log-loss 0.6236** 95% CI [0.6070, 0.6387] vs coin 0.693, **beating a material-sign heuristic**, calibrated to ECE 1.6%, **CI [0.548, 0.583]\*\***. The *live board is predictable even though the pre-game sheets are not* — the thesis, demonstrated. PORY is wired into KADABRA as a per-turn "you're at X%". Output: `data/pory.js`; report `data/pory-eval.json`.

### 4.4 CHOMP-EV — do CHOMP's brings beat humans'? (honest NULL)

The winnable team-preview test. For each held-out game (both full sixes, both actual brings, the winner), `engine/chomp_ev.js` ranks each side's *actual* bring among all 15 candidate brings by CHOMP's exact-damage coverage, and asks whether that quality signal tracks who won. On **1,205 games**: CHOMP's bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). It is **robust to forfeits** (0.505; a forfeit is usually a concession from a losing position, and dropping all forfeits does not change the result), and a measured **selection audit** shows the required "all four revealed" filter is a mild bias (eval 6.5 turns / 1280 rating vs 6.08 / 1267 excluded) that, if anything, *favours* CHOMP — making the null conservative. A **belief-weighted** variant (coverage vs the opponent's likely-4) also ties the coin (0.6924). Interpretation: the bring decision sits at the same near-coin ceiling as pre-game prediction; CHOMP's damage math stays validated and useful as a calculator, but "CHOMP builds better brings" is not yet empirically supported. This negative is a guardrail: it stops optimising a bring metric that carries no held-out winning signal. Report `data/chomp-ev.json`; test `tests/test-chomp-ev.js`.

**Multiplicity, corrected 2026-07-31.** The fit reports a 95% interval for all 56 features, so at alpha 0.05 about **2.8 of them clear zero by chance alone**. The family is **every feature in the shipped fit**, because every one is reported to the reader — choosing a smaller family after seeing which are large is the practice the correction exists to prevent. Uncorrected, **53** clear zero. Under **Benjamini–Hochberg** (FDR, 1995) **53** survive; under **Bonferroni** (FWER) **49**. Nothing significant uncorrected fails the FDR correction, so the headline count is not an artefact of having looked at 56. Computed by `engine/weight_multiplicity.js` →

`data/weight-multiplicity.json` . This says which weights are distinguishable from zero. It says nothing about whether an imitation-fitted weight is evidence about WINNING — a separate and larger question this project has measured going the other way.

**A phrasing the filter itself mandates.** `require_full_bring` conditions on game length: measured 2026-07-31, the games it keeps are **1.71x longer** on average (7.4 vs 4.3 mean turns; 19,589 kept vs 8,713 dropped). Every bring statistic in this project is therefore "*the bring, among games long enough to show it*", which is not the same as "the bring". `data/quality-filter.json` states this at the point of filtering and requires it to be said downstream; this is that.

#### 4.5 SLOWKING — team-preview Nash and the playstyle cycle (suggestive)

`engine/slowking_preview.py` solves a matchup matrix to a mixed-strategy equilibrium and grades it by **exploitability** (the worst-case win-edge a best pure counter extracts; lower is better; Nash  $\approx 0$ ), against greedy "single best deck" and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Over GURU's 13 species-archetypes the equilibrium is far less exploitable than uniform (Nash  $\approx 0$  vs 0.109), but greedy  $\approx$  Nash because this meta is currently near-transitive (a dominant deck). A **playstyle** re-analysis ( `engine/playstyle.js` classifies each team as TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) surfaces a non-transitive cycle — **TrickRoom**  $\rightarrow$  **HyperOffense**  $\rightarrow$  **Sand**  $\rightarrow$  **TrickRoom** — with a point exploitability gap of  $\sim 0.073$  for greedy; the equilibrium now correctly leads with Sun ( $\sim 31\%$ ), since Reg M-B Charizard is Mega-Y (Drought) and is classified as a Sun setter. **Honest caveat:** each cycle leg rests on only 13–18 games (win rates 73% / 71% / 67%) with 95% CIs that cross 50%, so the cycle is a **suggestive pattern, not a settled fact**; it will sharpen as the store grows. Where matchups *are* well-sampled they tend to run flat against intuition — **Rain vs Sun is 51% (n=236)** and **Tailwind vs no-Tailwind is 47% (n=756)**, both statistical coin-flips. Reports `data/slowking-eval.json` , `data/slowking-playstyle-eval.json` ; test `tests/test-slowking.py` .

## 5. Mathematics

**Wilson score interval** (used for every matchup rate) for  $w$  wins in  $n$  games,  $z = 1.96$  :  $(\hat{p} + z^2/2n \pm z \cdot \sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) / (1 + z^2/n)$  , with  $\hat{p} = w/n$  . It is well-behaved at small  $n$  and near 0/1, unlike the normal approximation.

**Value net.** Features  $x = [\text{alive\_diff}, \text{hp\_diff}, \text{my\_alive}, \text{foe\_alive}, \text{turn}/10]$  are standardised by train mean/std;  $P(\text{win}) = \sigma(w \cdot z + b)$  . Graded by held-out **log-loss**  $-(y \cdot \ln p + (1-y) \cdot \ln(1-p))$  and **Brier**  $(p-y)^2$  ; the coin scores  $\ln 2 = 0.6931$  and  $0.25$  respectively.

**Discrete choice — the scoring bot's policy (v3.28.0).** A player facing a turn chooses one of the legal (move, target) pairs. Writing  $x_j$  for the attributes of alternative  $j$  — type effectiveness against that specific target, base power, whether the move is already dead on the board, and the behaviour clone's  $P(\text{move} \mid \text{species})$  — the conditional logit model (McFadden 1974) is

$$P(\text{pick } j) = \exp(w \cdot x_j) / \sum_k \exp(w \cdot x_k) ,$$

with  $w$  estimated by maximising  $\sum_i [w \cdot x_{\{i, \text{chosen}\}} - \ln \sum_k \exp(w \cdot x_{\{i, k\}})]$  over **146,910** real human decisions from **6,091** clean open-sheet games (117,824 train / 29,086 held out), with a feature vector that has grown from 12 to **53**. The weights are **estimated, never written down**, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because a diagnostic stops being evidence once it becomes the objective.

Held out **by game** (decisions within a game are correlated — the same clustering argument as the CIs below): logL/decision **-1.6006** and top-1 **33.6%**, against the behaviour clone alone at  $-1.9302 / 27.1\%$  and uniform at  $-1.7627 / 24.1\%$ . Open-sheet games are used because they are the only corpus in which the **choice set** is

known rather than guessed: a normal replay reveals only the moves that were *used*, so alternatives reconstructed from revelation are biased by revelation.

The model's known limitation is **independence of irrelevant alternatives**: logit implies the odds between two options are unaffected by what else is on the menu, which fails for close substitutes (the red-bus/blue-bus problem). A set carrying two moves of the same type is exactly that case. Nested or mixed logit is the remedy and neither is implemented, so the fitted probabilities are a good ranking and only an approximate distribution.

**Equilibrium and exploitability.** Each preview is a two-player zero-sum matrix game on an antisymmetric edge matrix  $M[i,j] = (p(i>j) - p(j>i))/2$ . Regret matching (Hart & Mas-Colell) converges to an  $\epsilon$ -Nash. For a strategy  $x$ , **exploitability** =  $-\min_j (x \cdot M[:,j])$  — the worst-case loss to a best response; the Nash value is 0, so a Nash strategy scores  $\approx 0$  and a predictable single-deck strategy is punished.

**Confidence intervals.** Because per-turn states within a game are correlated, CIs are **bootstrapped by resampling games** (clustered), not states. Matchup-matrix uncertainty is propagated by **Beta(n·p+1, n·(1-p)+1) resampling** of each cell before re-solving.

**Future rating math.** For any descriptive meta-rating we will use an **intransitivity-capable** class (blade-chest / low-rank bilinear, Chen & Joachims 2016; or Nash-averaging, Balduzzi et al. 2018) and a **Helmholtz–Hodge / HodgeRank** decomposition (Jiang, Lim, Yao & Ye 2011) to split the matchup flow into a transitive ranking plus a cyclic (rock-paper-scissors) component — the correct tool for "which cores beat which" and for quantifying how cyclic the meta really is.

## 6. Limitations and honest ceilings

1. **The game-winner ceiling is permanent** (Elo  $\approx$  coin). SLOWKING/ALAKAZAM are judged on decision quality and self-play/ladder win-rate, never on match-outcome prediction.
2. **Revealed sets are partial** (a mon that never attacked reveals no moves); belief is a lower bound.
3. **Small samples in the meta layer.** Playstyle and core matchups are thin; those results are suggestive until the store grows.
4. **Policy is the residual GIGO — and in 3.28.0 the binding constraint is the OBJECTIVE, not the knowledge.** The damage is validated. The policy now runs a real damage calculation and does decide switches — both were listed here as missing and both became false, and they are corrected rather than quietly dropped. What remains: **one ply**, no model of the opponent's move, no search.

The sharper limitation is measured. Over 2026-07-30, **four separate feature additions produced four nulls**, while **two changes to the objective produced two large wins** — greedy action selection at +12 points (79.7% of decisive pairs) and self-play policy improvement at 55.9%. An overdispersion check across teams ( $\sim 1.00$ , against 1.169 for a known real effect) rules out the obvious confound, so the nulls are genuine. Adding knowledge to an imitation-fitted policy has stopped paying.

**RECONCILED 2026-07-31.** That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.

The cleanest demonstration is the pair-scoring layer (DODUO), which is **built, wired, controlled and measured, and loses at 42.0%** [39.9, 44.3] over 1,934 seed-paired games against its own zeroed control. Its fit prices "use a spread move beside my own ally that does not hurt it" at **-5.054** — a statement that humans rarely click it, not that it is bad. Refitting those weights for *winning* rather than *resemblance* is untested and is the project's top open question.

**CORRECTED 2026-08-01, and this paragraph should no longer be cited as it stands.** *The -5.054 was not a statement about human preference. fit\_joint.js matched a human's click by requiring the candidate's target to match, and a spread move is built with no target because it is not aimed — so no spread click could ever match. Spread moves are 14.94% of all human move clicks and 99.7% of them were thrown away; the fit used 24,997 of 82,483 joint turns, and the discarded 70% was exactly the turns containing the play the feature describes. Refitted, the weight is +0.863, and the corrected vector beats the shipped one at 66.7% and 65.9% of decisive pairs on two disjoint seed blocks. DODUO's 42.0% was measured on the contaminated vector and does not describe the current one. The imitation-versus-winning argument stands on its other evidence — greedy action selection is worth about 12 points — but not on this example.*

A separate class of defect, worth naming because it is not a modelling disagreement: a fact reaching one consumer and not the next. Priority blocking lived in the tag artifact and never reached the simulator, so **Sucker Punch beat a Farigiraf in every rollout ever run.** A switch-in's own ability never reached the code that chooses the switch — over 40,001 matchups, declaring Intimidate, Drizzle or Drought moved the feature vector in **zero** of them against a control's 2,754. And every mega forme carried a null ability, an empty moveset and no item, so **26.0% of the format's usage scored as threatening nothing.** All fixed; a gate ( engine/artifact\_audit.js ) now compares derived artifacts against their sources, because nothing had.

Every build\_lab win rate on record was measured against the older board-blind pilot and none has been re-run, so all of them remain provisional.

1. **Champions rule specifics** (sleep/paralysis edge cases) are flagged, not yet fully modelled.
2. **A result that does not record its own configuration is not reproducible, and three of the four rollout gates were in that state.** This is a methodological limitation, added 3.33.0, and it cost a published result. The R1 gate reported "68.18% against material's 65.26%, +2.91 [1.79, 4.04]"; recomputed from the only committed evidence it is **+0.456, 95% CI [-0.717, +1.630] — UNDECIDED.** No number was falsified. The row dump recorded {gid, turn, p, mpy, y, aliveDiff, hpDiff} and no sample size, no exploration rate and no build digest, so a dump taken at explore=0 and a dump taken at explore=1 were byte-compatible while differing by nearly four accuracy points. Only the surviving calibration shape distinguished them, in hindsight.

Auditing the other rungs against the same standard produced two further findings. **The R3 divergence gate publishes 72.9% over 70 decisions and records no control.** Its own script computes the quantity that makes a divergence rate mean anything — the same search on a different seed disagreeing with *itself*, whose true value is 0 by construction — writes it to standard output, and does not store it; the script's verdict branches on that comparison, so the artifact cannot state which branch its own run took. At a rollout budget of N=20 that floor measured *higher* than the divergence it was meant to validate. **The R2 cost gate timed a leaf the system does not run,** inheriting library defaults of explore=0 and a 20-turn horizon while the deployed leaf uses explore=1.0 at 60 turns.

The general statement is that a search is worth exactly what its leaf is worth, and a leaf measurement is worth exactly what its configuration record is worth. Every gate artifact now carries a sidecar ( engine/run\_stamp.js ) recording the budget, the exploration rate, the horizon, content digests of every source the gate reads, the

commit, and whether the working tree was dirty at the time. Artifacts predating the standard carry a stamp reconstructed from the commit that contained them, labelled as inferred rather than observed on every field.

1. **The headline metric has no current value, and that is the largest limitation in this paper** (added 3.59.0). §0 makes exploitability the number this project is judged on, and `data/exploitability.json` is declared void: the 2026-07-26 figure was fitted on 17 features against the 58 shipped, on an engine 25 wire-fixes old and before the quality filter existed, and the 2026-08-04 re-run had `data/policy-weights.json` — the defender itself — refitted at 22:15:24 UTC while it was running. **So the comparison with VGC-Bench's ~100% is a comparison we have set up and not yet made.** Producing one figure requires training a best response against a frozen agent, which is expensive, and it requires the frozen-release discipline to hold for the whole run — the 2026-08-04 void *was* an exploitability run, so this is a demonstrated failure mode rather than a hypothetical one.
1. **Two speed readings of the same engine differ by an order of magnitude and nothing caught it** (added 3.59.0, §3.0). 3,401, 1,606 and 13,041 are three measurements of MEDICHAM's throughput taken over two weeks; the first two are battles/sec and the third is turns/sec, and no ratchet, test or artifact compares any of them. A project whose central architectural decision rests on a speed ratio should measure that ratio the way it measures a win rate. It does not, yet.

## 7. The road to ALAKAZAM

---

ALAKAZAM is the in-battle capstone, built last on the inputs above. Given a live position it will output the win-%-optimal move (a mixed strategy) and its value by: (1) a **belief** over the opponent's hidden sets (XATU), updated by a Bayesian filter; (2) **depth-limited search** over the validated damage engine, solving each simultaneous turn as a **matrix game** (regret matching — this removes the speed bias that inverted the greedy engine); (3) a **learned value** at the leaves (PORY, grown to an NNUE-style net); (4) **human-anchoring** (KL-regularised to the behaviour-clone) so it stays strong and unexploitable. Inference is light (CPU / Web Worker / WASM); the strongest version needs offline RL on millions of human + self-play games and a rented cloud GPU. It is judged on decision quality and self-play/ladder win-rate with CIs — never on predicting the winner. A self-play data engine (MEW) is the pacing item toward the millions of games that path needs.

**Sequenced by the four phases (3.59.0, §0.3).** ALAKAZAM as described above is phase 3. It is reached only through phase 1 (MEDICHAM complete — a search needs an engine that is both fast and correct) and phase 2 (the gate: MILTANK untimed against MILTANK on the clock, ROADMAP #62, which decides whether compute buys anything at all). If phase 2 says no, phase 4 replaces this road with VGC-Bench's: behaviour cloning plus PPO under self-play, fictitious play and double oracle. That branch is approved in advance and is a result about the game, not a defeat — and it is cheap because the method is published and reproducible.

## 8. References

---

1. Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
2. Lanctot et al., *Monte Carlo Sampling for Regret Minimization* (MCCFR), 2009.
3. Moravčík et al., *DeepStack*, Science 2017. · Brown & Sandholm, *Libratus*, Science 2018.
4. Brown et al., *Combining Deep RL and Search* (ReBeL), NeurIPS 2020. · Schmid et al., *Player of Games*, 2021.
5. Angliss, Cui, Hu, Rahman & Stone, *VGC-Bench: A Benchmark and Strategy Suite for Competitive Pokémon Doubles Battling*, AAMAS 2026, [arXiv 2506.10326](https://arxiv.org/abs/2506.10326) — the only published work in this exact format; the source of the ~100%-exploitable finding and of the professional-beating result quoted in §0. ·

- Grigsby, Xie, Sasek, Zheng & Zhu, *Metamon* (offline RL + large sequence models, no search), RLC 2025, [arXiv 2504.04395](#). · Full survey of the field, with what each project implies for this one: [docs/PRIOR-ART.md](#).
6. Perolat et al., *DeepNash / R-NaD* (Stratego), Science 2022. · Vinyals et al., *AlphaStar*, 2019.
  7. Meta FAIR, *CICERO / piKL* (human-regularised RL, Diplomacy), Science 2022.
  8. Chen & Joachims, *Modeling Intransitivity in Matchup Data* (blade-chest), WSDM 2016. · Balduzzi et al., *Re-evaluating Evaluation* (Nash-averaging), NeurIPS 2018.
  9. Jiang, Lim, Yao & Ye, *Statistical Ranking and Combinatorial Hodge Theory* (HodgeRank), 2011.
  10. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, JASA 1927.
  11. McFadden, *Conditional Logit Analysis of Qualitative Choice Behavior*, in Zarembka (ed.), **Frontiers in Econometrics**, Academic Press 1974 — the discrete-choice model the scoring bot's policy is fitted with (§5).
  12. the Smogon damage calculator — community damage ground-truth. · Pokémon Showdown replay API.

---

**Companion documents.** [Slide deck](#) · [Technical documentation](#) · [Model ledger](#) · [Changelog](#)

---

## The role family: multi-label composition, WAR, and emergent roles (v2.6.0)

---

### Motivation

The earlier playstyle model assigned each team exactly one archetype. This is a **multi-class** framing of a **multi-label** object: a real team is Sun *and* Tailwind *and* Fake Out at once. Forcing one label discards most of the information and shatters the data into archetype×archetype cells of  $n \approx 11-18$ , which is why those matchup numbers were untrustworthy. The literature is explicit: multi-label classification (Tsoumakas & Katakis 2007), team-as-mixture-of-latent-roles (topic models; Blei-Ng-Jordan 2003), and latent roles beating raw identity for outcome prediction in team sports (arXiv 2304.08272).

### Role tagging (leak-free, data-earned)

We define 26 functional roles. A **species earns a role from data** — it is credited once it is observed performing the role ( $\geq 2$  times) across the store. Multi-effect moves carry several *factual* roles (Matcha Gotcha = special+heal+status; Body Press = wall+attack; Fake Out = tempo, not attacker). Role *presence* is binary; graded *strength* is deliberately **not** hand-set (asserting weights violates the project's measurement standard). A team's role vector is built from the **team-preview six**, which are public in every closed-sheet game, so the representation is uncensored and non-leaking.

Each ordered role pair (a, b) aggregates outcomes across every game where one side had a and the other had b, with a Wilson score interval. Because roles co-occur, each game contributes to many cells, so the **median cell rises from  $n \approx 15$  to  $n = 20$  across 1,051 cells (measured 2026-07-25 on 1,061 quality-filtered games)**. The figure of 7,971 published in v2.6.0 was retracted in 2.7.0 as an artifact of over-tagging (19.6 of 26 roles per team); it has since gone  $7,971 \rightarrow 95 \rightarrow \sim 50 \rightarrow 20$  as the taxonomy sharpened and the games were filtered — the structural fix. Empirically, however, a logistic model on the preview role-difference vector predicts the winner at held-out log-loss 0.694 vs a coin's 0.693: **roles describe and attribute, they do not predict**. The per-role coefficients are read as **win-credit per role**; KO-credit per species is measured directly from the turn log.

## WAR — Wins Above Replacement (species RAPM)

To attribute wins to individual Pokémon while controlling for teammates and opponents, we use basketball's **Regularized Adjusted Plus-Minus**. With one row per game, label  $y = 1$  if p1 won and features  $x_s = 1[s \in p1\text{ six}] - 1[s \in p2\text{ six}]$ , a ridge logistic regression yields  $\beta_s$ , species  $s$ 's adjusted win contribution. Ridge shrinks rare species toward zero. With replacement  $\beta$  at the 20th percentile and the logistic slope  $1/4$  at  $p = 0.5$ ,

$$\text{WAR}_s = 0.25 \cdot (\beta_s - \beta_{\text{replacement}}) \cdot (\text{games } s \text{ appeared}).$$

Held-out, the species model reached log-loss 0.6875 against a coin's 0.6931 — a result now **withdrawn 2026-07-25** — that figure was measured on the UNFILTERED store; on quality-filtered games WAR scores 0.7048 against a coin's 0.6931 (accuracy 0.502). The apparent signal was four bot accounts playing one team in 1,446 games. It does not beat a coin: *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not. Leaders are Basculegion, Kingambit, Sylveon; trailers are negative. Effect sizes are small and magnitudes ridge-shrunk — reported as an exploratory ordering, not settled wins.

## Emergent roles by NMF

Rather than hand-declaring roles, we factorize the data with **Non-negative Matrix Factorization** (Lee & Seung 1999):  $X \approx WH$  with  $W, H \geq 0$ , so each team is a non-negative **blend** of latent roles and each role is a recipe over features. Two cuts: (1) the team×move usage matrix (usage-weighted, which down-weights the closed-sheet censoring skew) recovers **offensive cores** but is dominated by attacking moves (relative reconstruction error 0.79); (2) the team×role matrix recovers **six clean archetypes** (error 0.53):

Intimidate+Fake-Out control, physical offense, special offense+sustain, bulky wall+screens+ redirection, Tailwind+Encore, priority. A move's loading on a role is **learned, not typed** — this is the principled source of graded primary/secondary strength (Label Distribution Learning, Geng 2016). The rank and the human names are the only non-data choices. Reconstruction error is **not** comparable across weightings; the correct model-selection criterion is **topic coherence** (Mimno et al. 2011), noted as the next refinement.

## Honest limits

Preview-composition signal is small; role-level winner-prediction ties a coin and WAR barely clears it. Role tags are a censored lower bound on capability (closed sheets reveal only used moves). NMF factors are soft and attacker-dominated at the move level. None of these is hidden; each is reported with its baseline.

## The coverage job lands, and the plan that drives it is amended (3.40.0)

Three results, each read from its artifact.

**The engine.** Wires 82–89 landed: the pre-turn shield class (Focus Punch / Beak Blast), the variable-power family, per-hit reactors, priority blocking across every move kind, Memento, drain-before-contact-toll order, the Steel Roller terrain gate, and secondary chances read from the FORMAT's rulebook with a drift counter.

`data/mechanics-census.json` moved 167 → **181 live of 186 probed, 5 missing with reasons**; the interaction matrix moved 68 disagreements → **13** (1,012 live carrier × reactor cases, 999 agree, 98.7%); the Showdown damage differential stands at 1/150, and the one row is a documented harness-layer artifact (Disguise), not an engine defect. Every new probe was demonstrated red against a deliberately broken in-memory engine before its green was believed. Shell Trap, flagged as entirely untagged, is `isNonstandard: 'Past'` — banned in this format; the missing tag is the format door working.

**The two-rulebook question, measured before it was architected.** `data/tags.json` and `CHOMP/data/move-effects.json` state overlapping move facts. Compared field by field (`data/rulebook-collision.json`, a ratchet that may fall and never rise): 151 comparable facts, 149 agree, **2 clashes** — and the

live one was Iron Head's flinch, where the tags copy carried the Champions format's 20%, the generic copy carried 30%, and the engine read the wrong one. Wire 89 closes it at the consumer. The unified-generator layer of the coverage plan is therefore insurance rather than urgency; the real exposure is the 193 facts (27 tag-only, 166 fx-only) the comparison cannot yet reach.

**The fitting gap below is now half closed.** The sheet-channel section that follows reported that the fit discarded the ability and moves the live player sees; the decision it asked for was made (open team sheets always, closed sheets deferred), and the single-move layer (MAG) is refitted on all four channels: 232,815 usable decisions of 241,927 seen at 3.42.0 (231,722 at the 3.40.0 fit, before the click-censoring pass removed 1,336 actions that were not clicks), with a point-of-use counter showing the declared channels reached the board on 99.67% of scored decisions — an environment match stated by measurement, not by diff-reading. The pre-refit weights are preserved and the two-channel incumbent is frozen as a release ( d3d04b669e18 ) for the pending paired held-out comparison against the 0.192-point noise floor. The joint (pair) layer is **not yet refitted**; until it is, the pair layer still prices against the two-channel board, and no improvement claim is made for either layer.

Separately, the coverage plan itself was re-examined at Will's request and amended where the re-examination found it wrong — mutation testing now precedes the handler registry (the original stub defense routed stubs into the one bucket the consumption ratchet deliberately never guards), mutation operators gained per-param perturbation and a derived-set rebuild hook, and the planned 58-dimension exploitability re-run is cancelled by measurement: a step-rule probe against a planted optimum showed one accepted step is worth 0.202 win-rate points against a 4.77-point resolution at the affordable budget, so the search moves to a 4–8-parameter reparameterization first. The full argument is docs/COVERAGE-PLAN-REVIEW.md . ABRA continues to have **no exploitability number**; data/exploitability.json remains void.

**Taunt was not implemented, and the largest disagreement by pair volume was an engine fault rather than a harness one (3.50.0).** The generated interaction matrix's disagreements were ranked by CARRIER uses x REACTOR uses — the pair's real frequency, not the carrier's — and the head of that list was Taunt , which appeared in twelve rows. The engine wrote the volatile, decremented it, and read it nowhere, so a Taunted body still landed Hypnosis, Decorate, Strength Sap and another Taunt; Showdown's two handlers ( onDisableMove at selection, onBeforeMove at execution) are now both wired off one derived table, volatile - > the move category it refuses . The row ranked #1 by volume, partingshot -> throatchop , had been filed as a probable staging artifact on the strength of a species mismatch; it was the engine — a pivot MOVE was given the bare-switch priority, so Parting Shot out-spiced everything and its replacement, not its user, took the incoming attack. Known disagreements fell from 94 to 72 (19 inside the scored set, 53 in buckets the gate discards). The census rose 211 -> 216 live of 219 probed; the damage differential did not move.

**The mutation tier's defect count was wrong in the direction that inflates it, and the correction is recorded rather than quietly applied (3.49.1).** The full sweep reported 97 DEFECT-CANDIDATE operators and an open total of 340; the two highest-usage rows were checked by hand and **both were false positives**. damageMultAll / lifeorb reads the tag for the damage and branches on the item's name only for the recoil — latent, not live. halvesDamage / lightscreen is not a defect at all: the engine ignores the tag's mult deliberately, because the artifact carries the singles value 0.5 and this is a doubles engine where the reduction is 2732/4096. A mutation verdict says what moved; it cannot see a deliberate override. The triage now grades every open operator **A/B/C/D from a parse of the frozen engine source** — A, no lookup for the tag and no branch on the carrier's name; B, the param is overridden by the engine's own constant; C, the behaviour is hardcoded by name; D, the param is read and this battery could not move it. **Nought of the 97 is class A.** The ratchet counts class A only (163 operators, 56 carrier × tag rows), because a number that counts false positives is a number people learn to ignore. Class A is not a count of missing mechanics — it says the fact reaches the simulator neither as a tag nor by name, and a third route ( mv.rc , data/move-effects.js ,

an action kind) can still carry it — so the census's `armed` field is the second sort key and 49 of the 56 rows have no armed probe. The rule is gated on three cases decided by hand before it existed (Taunt A, Light Screen B, Life Orb C) and refuses to publish if it cannot reproduce them.

## Outplayed turns are not noise: the click-censoring fix (3.42.0)

The policy fits learn from human clicks reconstructed out of replay logs. The log records what **happened**; the fit needs what was **clicked**. Across all **241,927 recorded human actions over 8,942 clean open-sheet games** ( `data/policy-weights.json` , the fit corpus), **1,336 (0.5522%) were never clicks at all** and were being fitted as though they were. The classifier that decides which is which is measured separately, against the protocol's own annotations over 9,230 games ( `data/click-censoring-census.json` , re-run 2026-08-05 on the current engine) — a slightly larger sweep than the fit, because the census reads every stored game while the fit takes only those it can build a board for:

| class      | n                | share          | mechanism                                                                   |
|------------|------------------|----------------|-----------------------------------------------------------------------------|
| CLEAN      | 236,712          | 94.9111%       | the recorded action is the click                                            |
| PARTIAL    | 3,328            | 1.3344%        | a redirector soaked the attack; the true target is one of two live foes     |
| COERCED    | <b>1,383</b>     | <b>0.5545%</b> | Encore replaced the action (1,152); a phazing move dragged the mon in (231) |
| unreadable | 7,126 + 824 + 31 | 3.2000%        | unmatched, trivial, ambiguous                                               |

The shares are the reason this table can be re-run without re-arguing the fit: the census has now been taken three times as the store grew, and the three class shares agree to a hundredth of a point across all of them ( `CHANGELOG.md` 3.42.0 and 3.47.0).

Both coerced classes were **invisible to every counter in the project**. The move Encore forces out is on the victim's own legal menu, so the matcher accepted it; and `|drag|` is stored with the same shape as `|switch|` , so a phased arrival read as a voluntary switch decision. This is label noise, and learning with mislabelled examples is strictly harder than learning with missing ones (Natarajan, Dhillon, Ravikumar & Tewari, 2013). It is also **Missing Not At Random** in Rubin's (1976) sense — the corruption lands precisely on the turns where the opponent's play worked.

Coerced actions now leave the labelled set and are counted. Redirected attacks are kept under the **partial-label** likelihood (Cour, Sapp & Taskar, JMLR 2011): the contribution is the marginal  $\log \sum_{c \in C} P_w(c \mid \text{board}, \text{choice set})$  over the candidate set, fitted by Generalized EM (Dempster, Laird & Rubin 1977; Neal & Hinton 1998), where the E-step is the responsibility  $q_c = p_c / \sum_{\{c\}} p_{\{c\}}$  and the M-step is the existing conditional-logit gradient on  $q$ -weighted rows.

**The estimator was validated on planted weights before the refit ran.** Real corpus feature rows, synthetic labels drawn from a known  $w^*$  , the real censoring process applied to those labels, three seeds ( `data/partial-label-em.json` ):

| regime                | rows censored | $\  \hat{w} - w^* \ _2$ oracle | naive         | EM            | noise floor |
|-----------------------|---------------|--------------------------------|---------------|---------------|-------------|
| heavy, systematic     | 20.96%        | 0.9978                         | <b>1.8913</b> | <b>1.0208</b> | 0.2600      |
| the corpus's own rate | 0.44%         | 0.9978                         | 0.9948        | 1.0021        | 0.2600      |

EM recovers **97.4%** of the censoring bias where the naive fit is visibly wrong, and at the rate the corpus actually censors the bias is **−0.0030 against a 0.2600 floor** — inside the noise.

**Result, paired on 48,274 held-out decisions over 1,851 games, bootstrapped over GAMES** ( `data/censoring-value.json` , re-run 2026-08-05 on the current engine and the grown corpus; every figure below is inside the interval of the smaller 3.42.0 run it replaces — that run's numbers are in `CHANGELOG.md` 3.42.0 and `docs/MEASURE.md` §14, and the comparison is tabulated in §17):

| held-out class                                                   | after – before                          |                           |
|------------------------------------------------------------------|-----------------------------------------|---------------------------|
| <b>COERCED</b> (n=293): P(model picks the action no human chose) | <b>−0.002613 [−0.003650, −0.001672]</b> | clears zero               |
| <b>PARTIAL</b> (n=650): mass on the true candidate set           | +0.000122 [−0.000261, +0.000514]        | contains zero             |
| PARTIAL: log-likelihood of the candidate set                     | −0.002662 [−0.004002, −0.001368]        | clears zero, <b>worse</b> |
| CONTROL, CLEAN (n=47,331): log-likelihood                        | +0.000485 [0.000189, 0.000777]          | clears zero               |
| CONTROL, CLEAN: top-1                                            | −0.008 [−0.107, 0.085]                  | contains zero             |

Read plainly: **the fabricated labels are unlearned and the redirection correction bought nothing measurable.** Its own validation predicted that — the class is 1.35% of actions with a candidate set of exactly two, so there was almost no bias to remove. No corpus-wide top-1 improvement is claimed; the fix's justification is that a wrong label is a wrong label. Every effect is smaller than its class's split-half noise floor and resolves only because the comparison is paired per decision.

The mechanism is legible in the refit: of 58 weights, 9 moved past 2 SE and the largest single movement is `stallIntoEncore` — *"I am about to Protect and something across from me can Encore me for it"* — at **−1.0502 → −1.6281**. The poisoned rows were victims "choosing" their last move under an active Encore; deleting them makes clicking into an Encore threat look worse, which is the direction the mechanic predicts.

**Three limits, stated.** (i) The two vectors also differ by 86 games of corpus growth and by the refit itself, so the attribution rests on the weight-movement pattern rather than on an isolated control; `CENSORING=off` exists to run that control and has not been run. (ii) Priority-blocked attempts (Armor Tail, Queenly Majesty) are recoverable — the protocol names the attacker and the move in **299 of 299 cases (100.0%)** — but live only in raw logs covering 67.23% of the corpus, and the missing third is one archive, so recovering them would reweight the sample by source. (iii) `board.js` narrows the choice set for a Choice item and not for the `onDisableMove` family, so **2,374 of 146,413 logged actions (1.6214%)** were priced against a menu that had already shrunk — a wrong denominator rather than a wrong label, and a separate refit.

## A degenerate signature: when every arm of a controlled probe returns the same integer (3.56.0)

The accuracy subsystem was probed with the instrument this division already had — stage the mechanic, stage its absence, run both, difference the result. Six arms over four mechanics, ~5,000 combined uses in the store:

| arm                                  | with | without |
|--------------------------------------|------|---------|
| Coil ( +1 accuracy )                 | 0    | 0       |
| Minimize ( +2 evasion on the target) | 258  | 258     |
| Wide Lens ( ×1.1 )                   | 0    | 0       |
| Bright Powder ( ×0.9 )               | 116  | 116     |
| Sand Veil, in sand ( ×0.8 )          | 115  | 115     |
| No Guard ( accuracy → true )         | 0    | 0       |

**Exact equality across every arm is a stronger signal than a wrong number, and it is a different one.** A miscalibrated modifier produces a difference of the wrong size; a difference of *identically zero*, repeated over four independently-implemented mechanics, is evidence about the **path**, not the **parameters**. Section *"A mechanic that fires everywhere"* (3.44.0) records the mirror-image signature — a rule firing on 100% of a population it should split — and both are instances of the same diagnostic: read the *distribution* of the controlled difference, not its mean.

Three unrelated defects were on that path, which is why no single hypothesis explained all six arms. (i) The Showdown→engine stat map sent `accuracy` and `evasion` to `null`; eleven boost appliciers key off that map, so a payload of `{atk:+1, def:+1, accuracy:+1}` applied two of its three components and reported success. (ii)

No item or ability was consulted for accuracy at any call site. (iii) The roll called `moveAccuracy(id, field)` — **a signature that admits no attacker and no defender**. Defects (i) and (ii) are omissions and are ordinary; (iii) is a *type-level* impossibility, and it is the one worth generalising from: a function whose parameters cannot express the question is unfalsifiable by any test that only checks its output. The census had graded the accuracy family LIVE on exactly that basis for as long as it had existed.

The repair is one authority, `hitChance(att, def, id, field, ctx)`, called at all four to-hit sites, with `printedAccuracy` preserving `true` (never-miss) as distinct from `100`, the standard  $(3+n)/3$  Gen-III+ stage table [Bulbapedia, *Accuracy*], and the roll relocated **below** target resolution so a defender exists to interrogate. Direction is not hand-typed: `ACCURACY-MODIFIER CONFORMANCE` re-derives all 12 handlers from the live format object and takes sign from the hook name — 12 handlers, 13 rows, **0 disagreements**.

Separately, `Substitute` deducted 25% of the user's HP through the generic `costsUserHP` path and created no substitute: `playerAction` resolves the move to `kind: 'affect'`, so the `kind==='sub'` branch added in WIRE 42 was unreachable at the time it was written. 1,976 clicks in the store of an action **strictly dominated by passing** — a rare case where the correct baseline is not "a slightly worse policy" but "a negative-value action no rational agent takes", which makes any policy fitted over those turns miscalibrated in a *direction*, not merely in magnitude.

The bypass rule is likewise derived rather than reasoned. The intuitive encoding — *sound moves pass through a substitute* — is true and insufficient: the three highest-usage bypassing moves in this format are **Encore (4,848), Taunt (1,503) and Disable (730), and none carries the sound flag**. `SUBSTITUTE-BYPASS CONFORMANCE` re-derives `bypasssub` across all 500 moves: **51 carried, 0 missing, 0 invented**. The general principle is the one this project states as *flags feed tags; match on tag shape, never on a name* — a semantic proxy for a mechanical flag will be right on the examples that motivated it and wrong on the tail that matters.

**Reported rather than closed**. Five tags are *absent*, not unprobed, and are declared with usage counts instead of being probed red — gate (c) ratchets on "every probe MISSING", so a red probe would have broken a ratchet to record a fact the (b) column already records. The largest, `ability|auraBoost` (5,663 uses), is a **representational** limitation rather than a missing branch: the multiplier is field-wide over the full roster and `dmgRange` is given two bodies and a field. Wiring it changes a `board.js`-facing input and is therefore a design decision, routed as one.

## A mechanic that fires everywhere is not a mechanic that works (3.44.0)

Psychic Terrain refuses priority moves. The simulator knew that, and had known it since the tag artifact was first read. What it did not know is that the refusal only applies to a **grounded** target — so `priorityRefusedAbove` applied the terrain bar *outside* its own defender loop, without inspecting a single body:

```
for (const d of (defenders || [])) { /* the ability bar */ }
if (field && terrainId(field.terrain) === 'psychic') out = Math.min(out, 0);
```

The cost is concentrated on the most-clicked move in the format. **Fake Out (12,872 corpus uses)**, along with Extreme Speed, Sucker Punch, Aqua Jet, Ice Shard and Upper Hand, failed against every Flying type, every Levitate body and every Air Balloon whenever a Psychic Terrain was up.

The expected behaviour was taken from the official engine rather than from anyone's memory. Playing Incineroar's Fake Out into a Psychic Terrain raised by the opposing Indeedee's Psychic Surge, at the pinned commit under `gen9championsvgc2026regmb`:

| target                  | official engine                         | damage    |
|-------------------------|-----------------------------------------|-----------|
| Garchomp, grounded      | -activate move: Psychic Terrain         | 0         |
| Orthworm, Earth Eater   | -activate move: Psychic Terrain         | 0         |
| Talonflame, Fire/Flying | -hint "doesn't affect airborne Pokémon" | 237 → 216 |
| Hydreigon, Levitate     | -hint "doesn't affect airborne Pokémon" | 251 → 233 |
| Talonflame + Iron Ball  | -activate move: Psychic Terrain         | 0         |

Two findings sit underneath the fix and matter more than it does.

**The predicate existed three times, and none of the three was the one that mattered.** Grounded-ness was written by hand in the entry-hazard block, in the switch-trapping branch, and in the Grassy Terrain heal — three copies that disagreed with each other about Iron Ball and about Eelevate. This is the failure mode CLAUDE.md names *facts are global*: the Grassy Terrain copy applied only the type half of the rule and **counted its own known-wrong half** in a failure counter. Somebody knew it was wrong, declared it, and the declaration outlined the reason for it — the derivation it said was unavailable had landed a release earlier. One `isGrounded(mon)` now answers the question for all four sites.

**The census could not see the defect, because a scope is not a knob.** The existing probe for this mechanic stages the block against a Garchomp, which is Dragon/Ground. It passes on the broken engine and on the fixed one. Every instrument in this division asks whether a mechanic *fires*; none asks whether it fires *only where it should*, and this is the fourth defect of that shape in two days. The replacement probe carries five arms, and the reason for each is that a smaller probe would have passed on some specific wrong engine — including an **Earth Eater** arm, which is the reason the airborne ability set is a name rather than a shape read. The tempting artifact shape, `typeImmunity {type: 'Ground'}`, has three members: Levitate, Eelevate **and Earth Eater**. Orthworm is immune to Ground and firmly on the floor, and the official engine says so. Mechanics census **210 live of 213 probed**, 3 missing with written reasons, 0 hollow.

## The matrix's own arithmetic is closed, and the coverage figure moves (3.43.0)

The interaction matrix is this project's largest conformance instrument, and until this release nothing checked its arithmetic. It printed a theoretical cross product, an emitted count and a ledger of named drops four lines apart, and **no code compared them**. They did not agree.

The identity is `theoretical === staged + dropped`, per axis, and it is now asserted at generation time rather than printed for a reader. It found three defects on its first run:

- The denominator omitted the generator's own supplementary keys.** `tests/interaction_matrix.js` stages against `tags.linkage MERGED` with keys it derives itself; the theoretical total counted only the artifact's. 170 pairs were staged or dropped against a universe that had never heard of them. Theoretical **8,506 → 8,676**.
- The type axis mis-costed its depth-cap tail by one.** The index was incremented before the cap was tested, so the tail excluded the very carrier the break was rejecting — 32 firings, 32 pairs of silence, in the direction that *flatters* the coverage rate.
- The outcome buckets were not a partition.** `saturated` did not exclude a case that had thrown and `ko_timing` excluded nothing, so four cases were counted twice and the five printed totals summed to more than the number of cases run.

With the ledger closed, the generator recovers pairs it had been dropping unnamed: emitted **1,514 → 1,675**, live **899 → 1,031**.

**The headline agreement figure falls, and that is the instrument working.** The matrix reports **1,027 of 1,031 (99.6%)**, where the previous release reported 899 of 899 (100.0%). MEDICHAM did not regress: the four disagreements — Shield Dust against Fake Out, Throat Chop and Psychic Noise, and Steadfast against Upper Hand — sit on pairs the smaller generator never emitted. A 100.0% computed over a denominator that silently dropped 5,090 pairs was the less honest number. The four are `UNWIRED` rather than miscalculated: MEDICHAM's own two arms are identical on each, meaning the knob is absent rather than wrong.

The self-test is the point. `--selftest-reconcile` mis-costs exactly one drop by one pair — the smallest lie the ledger can tell — and requires the identity to stop the run. The file previously carried a header stating that the assertion fires; the assertion was defined and never called.

## Layer 0 executes; the joint layer refits; the channel value is measured (3.41.0)

Same night, three division runs later. **Engine:** Layer 0 of the coverage plan is done — census **202 live of 205 probed, 3 missing with reasons**; interaction matrix **100.0%** (899 cases after retiring four redundant tags shrank the generated set from 1,012 — the retired facts live on under their surviving tags); the DEAD-tag ratchet fell **61 → 38**; the 26 orphan ability/item tags are triaged with a full disposition table in ENGINE.md. Two real bugs surfaced in passing and were fixed with probes shown failing first: the Intimidate retaliation arithmetic (Defiant read net +2 where the game gives +1; Competitive skipped the Attack drop) and Sheer Force missing its  $\times 1.3$  while its secondary-suppression half worked — strictly worse than no ability. The mutation tier's injection point ( `__setDB` plus the derived-set rebuild hook the amended plan requires) landed and was exercised 26 times by the probe-red-demonstration harness.

**Measure:** the JOINT layer is refitted on the four-channel sheet — 95,886 usable joint turns, channel-reach counters at 99.7%, feature semantics verified — closing the second half of the fitting-environment gap. The held-out channel-value measurement ran A/B/C against the frozen two-channel incumbent (release `d3d04b669e18` ), 44,982 paired decisions, 10,000 game-bootstrap resamples:

| paired difference                 | logL/decision                         | top-1 points            |
|-----------------------------------|---------------------------------------|-------------------------|
| information alone, weights frozen | <b>+0.002853</b> [0.001611, 0.004072] | +0.009 [-0.140, +0.157] |
| refit, given the information      | <b>+0.002234</b> [0.001638, 0.002831] | +0.165 [0.029, 0.299]   |
| everything vs what shipped        | <b>+0.005087</b> [0.003854, 0.006331] | +0.173 [-0.011, +0.360] |

Split-half noise floor of the shipping arm: **0.331 top-1 points** (median, 20 cuts). The honest reading: the sheet channels buy a real per-decision likelihood gain — every logL interval clears zero — and **no demonstrable top-1 gain**; the one clearing interval is half the noise floor and resolves only because the comparison is paired. The first measurement attempt self-voided when the engine moved mid-run and was re-run clean — the release discipline working as designed.

**And the tags regeneration was gated the way the rules demand:** after the staged derivations landed in `data/tags.json` , `feature_fixture --check` confirmed both fitted weight vectors still agree with `board.js` on every fixture board — the new tags moved zero of the 58 feature columns, so the night's fits stand unre-run.

## Measuring an engine that is being edited (3.36.0 – 3.39.0)

### A refit that bought nothing, reported as such

The feature function was wrong about the weather on 10.72% of turn-boards: `engine/board.js` carried a private weather map that recognised Desolate Land and Primordial Sea — neither of which this format can produce — and did not recognise **sandstorm or snowscape**. Routing both reads through the engine's own

exported `weatherId` moves **14 of 58 feature columns**, 1,768 of 234,873 vectors (0.75%), 892 of 32,054 decisions (2.78%), and touches 238 of 1,200 games (19.83%) — consistent with the 18.5% sand/snow census. Paired per decision on the same 46,162 held-out decisions across 1,772 games, bootstrapped over 10,000 game resamples:

| paired difference               | logL / decision                       | top-1 points                 |
|---------------------------------|---------------------------------------|------------------------------|
| fix alone, weights frozen       | <b>+0.000348</b> [0.000075, 0.000623] | <b>+0.048</b> [0.009, 0.093] |
| the refit, given fixed features | -0.000076 [-0.000172, +0.000021]      | -0.074 [-0.155, +0.004]      |
| everything vs what shipped      | +0.000273 [-0.000010, +0.000556]      | -0.026 [-0.117, +0.064]      |

Split-half noise floor for the refitted arm, 20 cuts: **median 0.192 top-1 points**. The fix is detectable *only* because the comparison is paired, and it is a quarter of that floor. **Refitting bought nothing** — the interval contains zero on both metrics, 1 of 58 weights moved beyond 2 SE, and the L2 of the whole weight change is 0.216. The fix was worth making because the feature function was wrong about the game, not because a metric improved; it did not need one and it did not get one.

**The fitting environment is not the playing environment, and the gap is 20× the defect above**

`engine/fit_policy.js:376` hands the board `{nature, item}`. `engine/magnemite.js:522` — the live player — hands it `{nature, item, ability, moves}`. Over 14,400 sheet entries in 1,200 games, **100.0% declare an ability and 100.0% declare four moves**, and the fit discards both.

|                     | weather defect | sheet-channel gap              |
|---------------------|----------------|--------------------------------|
| vectors that move   | 1,768 (0.75%)  | <b>37,460 (15.95%)</b>         |
| decisions that move | 892 (2.78%)    | <b>16,177 (50.47%)</b>         |
| feature columns     | 14 of 58       | <b>20 of 58</b>                |
| games touched       | 238 (19.83%)   | <b>1,197 of 1,200 (99.75%)</b> |

The choice set is identical game for game, so this is purely what the board *knows*. **Half of every decision the fit trains on is priced against a board the player does not see**. This is CLAUDE.md's fitting-vs-playing rule broken a second time and in the **opposite direction** from 2026-07-28 — the bot now sees *more* than the fit — which is precisely why nothing was watching for it. Not landed: it is one line plus a full refit, and it first needs a decision about the games where the opponent declines open team sheets, since a model fitted on four channels degrades differently from one fitted on two.

**Interactions, generated rather than sampled**

8,795 theoretical carrier × reactor pairs; 2,300 staged; **1,634 that can genuinely co-occur**, where co-occurrence is decided by the reference engine's own two arms differing rather than by our judgement — so "correctly blocked" stays distinguishable from "silently absent". The engine agrees on **1,614 of 1,634 (98.8%)**. Every pair the generator refuses is counted under a named reason and printed on each run. The 156 ordered persistent-field pairs each become an 8-turn script, which is the only construction that can observe *Trick Room was already up when Tailwind landed*; that axis went from 30/156 to **156/156**.

**Validity: a measurement reads a frozen release**

Three division agents ran concurrently with their files separated, and a 7,100-game exploitability run was still destroyed: the defender's own weight vector was refitted between the two legs, and the simulator showed four distinct content digests inside eight minutes. Nothing failed and nothing crashed.

The correction is not scheduling — serialising the divisions forfeits the parallelism they exist for. A measurement now opens an **immutable snapshot** ( `engine/engine_release.js` ) of the twelve files whose content can change a reported number, the weights included, and reads those bytes rather than the live tree. It

is a copy and not a checksum: verifying digests afterwards establishes only that the run was wasted.

`engine/provenance.js` correspondingly stopped deciding staleness by **mtime** — the method this project's own rules discredit by name — and now compares content digests, honours a self-declared `void: true`, and prints how many artifacts still rest on timestamps alone (**0 verified, 92 by mtime**), ratcheted downward. On its first run the content check caught a rollout artifact computed against a version of its own generator that had since changed.

**Consequently ABRA publishes no exploitability figure.** The prior 63.2% [56.6, 69.3] is retracted on its own merits — 17 features against the 58 shipped, an engine 25 wire-fixes old, computed before the quality filter existed — and the re-run is void. One figure from the void run survives, because both of its legs fall inside a single stable window: the mirror control at **49.7% [46.2, 53.2]**,  $n=782$ , which retires the concern that an earlier 47.5% indicated a seat or pairing asymmetry rather than noise at  $n=217$ . A separate finding stands independently of the invalid tree: the attack **dies in 58 dimensions**, accepting 1 of 24 hill-climb steps against 10 of 18 at 17 features, so the step rule needs correcting before the re-run is worth its cost.